

NORMA TÉCNICA Y GUÍA DE INGENIERÍA · ANTIGRAVITY AI

SPEC-DRIVEN DEVELOPMENT (SDD)

Marco de Referencia Metodológico, Arquitectura de Sistemas y Lecciones Prácticas de Implementación

Un análisis exhaustivo del ciclo de vida de desarrollo guiado por especificaciones ejecutables, compuertas de aprobación formal, diseño por contrato y desarrollo asistido por IA, fundamentado en el caso de estudio real de la plataforma **CV Wizard Open Source**.

EL PIPELINE CANÓNICO DE SPEC-DRIVEN DEVELOPMENT (SDD)

FASE 1

Especificación

01-specification.md

Gate 1

FASE 2 & 2B

Arquitectura & UI

02-arch.md + DESIGN.md

Gate 2

FASE 3

Plan de Tareas

03-tasks.md (Atómico)

Gate 3

FASES 4 Y 5

Código & Validación

src/ + RTM Matrix

Gate 4 & 5

PROYECTO DE REFERENCIA Y CASO DE ESTUDIO

CV Wizard Open Source (`practica-sdd-cv-maker`)

Stack: Astro 5, React 19, Tailwind CSS v4, Google Stitch Standard, TypeScript Estricto.

METADATOS DEL DOCUMENTO

Versión: 1.0.0 Oficial

Fecha: Septiembre 2026

Clasificación: Referencia Metodológica

Resumen Ejecutivo y Estructura del Marco

PROPÓSITO DEL DOCUMENTO

Este documento formaliza la metodología **Spec-Driven Development (SDD)** como marco de referencia técnico para la ingeniería de software moderna. Se fundamenta en la experiencia directa obtenida durante la construcción integral del proyecto **CV Wizard Open Source**, demostrando cómo una aproximación basada en especificaciones desacopladas y compuertas de calidad elimina la entropía del código, garantiza la trazabilidad del 100% de los requisitos y optimiza la colaboración entre ingenieros y agentes de inteligencia artificial.

Tabla de Contenidos

1. Introducción y Justificación del Paradigma SDD	Pág. 3
La crisis del "Vibe Coding", definición formal y los 5 axiomas fundamentales.	
2. Genealogía y Comparativa Metodológica Multidimensional	Pág. 4
SDD frente a TDD, BDD, DDD, Waterfall y Scrum tradicional. Sinergias operativas.	
3. El Ciclo de Vida Canónico de SDD (Fases 1 a 5 + 2B)	Pág. 5-7
Anatomía de artefactos, criterios Given-When-Then, compuertas de aprobación y Stitch.	
4. Estudio de Caso Práctico: Implementación de CV Wizard Open Source	Pág. 8-9
Análisis forense de arquitectura, desacoplamiento de plantillas, print engine y NFRs.	
5. SDD en la Era de la Inteligencia Artificial y Agentes Autónomos	Pág. 10
SDD como sistema de control de vuelo para LLMs, gestión de contexto y mitigación de alucinación.	
6. Marco Operativo: Checklists, Plantillas y Reglas de Oro	Pág. 11-12
Listas de chequeo por fase, matriz de trazabilidad y las 10 reglas inviolables.	
7. Conclusiones y Hoja de Ruta para Nuevos Proyectos	Pág. 13
Recomendaciones estratégicas para la adopción organizacional de SDD.	

Ficha de Control del Documento

Atributo	Detalle Técnico
Título del Estándar	Marco Metodológico de Desarrollo Basado en Especificaciones (Spec-Driven Development)
Versión y Estado	Versión 1.0.0 Oficial · Estado: ● Aprobado para Publicación y Referencia
Público Objetivo	Arquitectos de Software, Líderes Técnicos, Ingenieros Fullstack, Desarrolladores con Agentes IA
Repositorio Fuente	<code>/home/jotace/Proyectos/Práctica SDD/</code>
Normativa Asociada	IEEE 830 / ISO/IEC/IEEE 29148 (Requisitos), Google Stitch Design Standard, W3C WCAG 2.1 AA

1. Introducción y Justificación del Paradigma SDD

1.1. La Crisis del Desarrollo No Estructurado ("Vibe Coding")

Con la proliferación de asistentes generativos y Modelos de Lenguaje Grande (LLMs), la industria del software vive un dilema crítico: *la velocidad para generar líneas de código ha crecido exponencialmente, pero la mantenibilidad, consistencia arquitectónica y predictibilidad han colapsado.*

El desarrollo no estructurado o "Vibe Coding" (saltar directamente a codificar o a lanzar prompts informales) genera tres patologías recurrentes:

- **Desviación Silenciosa de Alcance (Scope Drift):** El software implementa suposiciones arbitrarias del desarrollador o del modelo de IA, distorsionando las necesidades reales del usuario.
- **Acoplamiento Prematuro y Deuda Técnica:** Se fijan dependencias y estructuras de datos antes de definir los requisitos no funcionales (rendimiento, privacidad, portabilidad).
- **Alucinaciones e Inconsistencias en Agentes IA:** Los LLMs pierden coherencia tras varias iteraciones si no disponen de un documento maestro que actúe como ancla cognitiva ("Ground Truth").

1.2. Definición Formal de Spec-Driven Development (SDD)

DEFINICIÓN CANÓNICA DE SDD

Spec-Driven Development (SDD) es una disciplina formal de ingeniería de software que establece que *ninguna línea de código de producción, prueba automatizada o configuración debe escribirse antes de que sus requisitos funcionales, arquitectura y tareas atómicas hayan sido especificados, versionados y formalmente aprobados en artefactos desacoplados.* En SDD, el código fuente es un **subproducto compilado y verificable de la especificación.**

1.3. Los 5 Axiomas Fundamentales de SDD

Axioma	Significado Metodológico	Manifestación en la Práctica
1. SSOT Inmutable (Single Source of Truth)	La especificación es la única fuente de verdad. Ante discrepancias entre código y especificación, la especificación manda.	Todo comportamiento no previsto en <code>specs/01-specification.md</code> se clasifica como anomalía o defecto.
2. Contratos Inviolables	Las interfaces de datos, firmas de métodos y esquemas de dominio se congelan antes de la lógica interna.	El archivo <code>src/types/cv.ts</code> se redacta y valida en Fase 2 antes de crear componentes React.
3. Trazabilidad Total	Todo requisito debe rastrearse bidireccionalmente: Requisito ↔ Arquitectura ↔ Tarea ↔ Código ↔ Prueba.	La tarea <code>TASK-2.2</code> declara explícitamente implementar el Criterio 3.1 de la historia <code>US-03</code> .
4. Compuertas Secuenciales (Approval Gates)	El paso de una fase a la siguiente exige la validación y firma explícita (humana o automatizada) del artefacto producido.	No se inicia el desglose de tareas (Fase 3) hasta que la Arquitectura (Fase 2) esté marcada con ● Aprobado.
5. Desacoplamiento del 'Qué' y del 'Cómo'	La intención del negocio se aísla completamente de las tecnologías particulares utilizadas para implementarla.	El modelo del CV se define como un schema conceptual antes de decidir librerías de interfaz o estado.

2. Genealogía y Comparativa Metodológica Multidimensional

2.1. Raíces Formales de la Ingeniería de Software

SDD se fundamenta en principios consolidados de la ciencia de la computación:

- **Diseño por Contrato (Bertrand Meyer, 1986):** Colaboración mediante precondiciones, postcondiciones e invariantes formales, elevados al ciclo de vida del producto completo.
- **Principio de Sustitución y Especificación Formal (Barbara Liskov, 1987):** Separación entre contrato abstracto e implementaciones intercambiables.
- **Estándares IEEE 830 e ISO/IEC/IEEE 29148:** Requisitos no ambiguos, verificables y consistentes.

2.2. Matriz Comparativa Multidimensional

Dimensión	SDD (Spec-Driven)	TDD (Test-Driven)	BDD (Behavior-Driven)	Waterfall Clásico	Agile / Scrum Informal
Artefacto Central	Especificación funcional y diseño técnico estructurado.	Suite de pruebas unitarias automatizadas.	Escenarios de comportamiento (Gherkin/Feature files).	Documento de requisitos exhaustivo cerrado.	User Stories en backlog o tickets de Jira.
Punto de Partida	Contrato funcional + Modelo conceptual + Gate.	Escribir una prueba unitaria que falle (Rojo).	Conversación entre Product Owner, Dev y QA.	Análisis exhaustivo inicial de meses.	Estimación en puntos de historia y codificación.
Control de Alcance	Muy estricto mediante Compuertas de Aprobación.	Moderado (restringido al alcance de los tests).	Alto (guiado por criterios de aceptación).	Rígido y burocrático (cambios caros).	Variable (alta propensión a scope creep).
Aptitud para Agentes IA	Máxima (Óptima): Contexto acotado, alucinación reducida.	Media: La IA a menudo escribe tests que validan bugs propios.	Alta: Los escenarios GWT guían la generación de código.	Baja: Demasiada sobrecarga documental monolítica.	Baja/Media: Falta de detalle técnico provoca alucinación.
Costo de Refactorización	Mínimo: Las interfaces y tipos están blindados.	Bajo en lógica interna; alto si cambian contratos.	Medio: Requiere actualizar escenarios y step-definitions.	Extremo: Requiere reiniciar el ciclo completo.	Alto: La deuda técnica acumulada frena el avance.

2.3. Sinergias Metodológicas: El "Ecosistema SDD"

SDD COMO METODOLOGÍA PARAGUAS

SDD no compite con TDD ni BDD; actúa como su marco orquestador natural:

- **SDD + BDD:** La Fase 1 adopta la sintaxis *Given-When-Then* para formular criterios de aceptación verificables.
- **SDD + TDD:** La Fase 3 (Desglose de Tareas) define pruebas de unidad que el desarrollador ejecuta bajo el ciclo Red-Green-Refactor en Fase 4.
- **SDD + DDD:** La Fase 2 establece el Lenguaje Ubicuo y los modelos de dominio que aíslan la lógica del negocio de los frameworks.

3. El Ciclo de Vida Canónico de SDD (Fases 1 a 5 + 2B)

3.1. Visión Holística del Pipeline

El ciclo de vida en SDD es estrictamente secuencial a nivel de compuertas de aprobación, pero iterativo y modular dentro de cada fase:



3.2. Fase 1: Especificación de Requisitos y Dominio (01-specification.md)

El objetivo de la Fase 1 es desentrañar el problema de negocio y fijar los límites del sistema con rigor matemático:

- **Visión y Propósito del Producto:** Justificación de existencia y propuesta de valor diferencial.
- **Historias de Usuario Formalizadas (GWT):** Formato estándar Given-When-Then para eliminar ambigüedades.
- **Requisitos No Funcionales (NFRs):** Rendimiento, latencia de renderizado, privacidad local y soporte de impresión.
- **Modelo Conceptual de Dominio (Schema Agnóstico):** Estructura canónica de los datos expresada sin acoplarse a librerías de UI:

```
{
  "profile": { "fullName": "string", "headline": "string", "email": "string", "phone": "string" },
  "experiences": [ { "id": "string", "company": "string", "position": "string", "startDate": "string" } ],
  "education": [ { "id": "string", "institution": "string", "degree": "string" } ],
  "skills": [ { "id": "string", "name": "string", "level": "basic" | "intermediate" | "advanced" } ]
}
```

GATE 1: CRITERIO DE APROBACIÓN DE ESPECIFICACIÓN

Para superar el Gate 1, el cliente o Product Owner debe validar que el documento responde al 100% de los casos de uso esperados y que los criterios Given-When-Then son verificables por un tercero sin requerir explicaciones verbales adicionales.

3. Ciclo de Vida Canónico (Cont.): Fases 2 y 2B

3.3. Fase 2: Arquitectura y Diseño Técnico (02-architecture-design.md)

La Fase 2 transforma los requerimientos abstractos en una solución de ingeniería robusta, justificada y escalable:

- **Matriz de Decisión Tecnológica (Trade-off Analysis):** Justificación comparativa de cada componente del stack, descartando alternativas inadecuadas.
- **Diagrama de Flujo de Datos Unidireccional:** Modela el ciclo de vida del dato desde la entrada en los formularios hasta su persistencia en `localStorage` y renderizado en el visor A4.
- **Contratos de Tipado Estricto de Dominio:** Interfaces TypeScript exhaustivas que actúan como ley inmutable para la codificación.

GATE 2: VALIDACIÓN ARQUITECTÓNICA

No se permite escribir código de interfaz sin que las interfaces TypeScript estén completamente definidas y validadas con el compilador.

3.4. Fase 2B: Sistema de Diseño e Integración Visual (Stitch / DESIGN.md)

Uno de los mayores focos de deuda técnica en desarrollo web es la improvisación de estilos CSS. SDD incorpora la **Fase 2B** utilizando el estándar abierto **Google Stitch** (`DESIGN.md`):

- **Tokens Ejecutables en YAML Frontmatter:** Paleta semántica, escala tipográfica geométrica, radios y espaciados estandarizados.
- **Compilación Automatizada a Tailwind CSS:**

```
npx @google/design.md export DESIGN.md --format css-tailwind > src/styles/design-tokens.css
```

- **Linting de Accesibilidad:** Validación estricta con `@google/design.md lint` para garantizar contrastes WCAG AA (ratios mínimos de 4.5:1 para texto normal).

Token de Diseño	Valor Stitch	Propósito Arquitectónico
<code>primary</code>	<code>#1e40af</code> (Azul Zafiro)	Identidad corporativa, encabezados de sección y acentos de plantilla.
<code>surface-sidebar</code>	<code>#0f172a</code> (Pizarra Profundo)	Fondo de la columna lateral en la plantilla moderna de CV.
<code>headline-lg</code>	Inter, 32px, bold, line-height 1.2	Nombre del candidato en el currículum, legible y jerárquico.
<code>rounded-md</code>	8px	Radio estándar para tarjetas de formulario, inputs y botones de acción.

3. Ciclo de Vida Canónico (Cont.): Fases 3, 4 y 5

3.5. Fase 3: Desglose Atómico de Tareas (`03-tasks.md`)

La Fase 3 divide la arquitectura en unidades de trabajo atómicas, independientes y cronológicamente ordenadas. Cada tarea incluye:

- **Trazabilidad Directa:** Referencia explícita a la Historia de Usuario de Fase 1 y componente de Fase 2.
- **Definition of Done (DoD):** Criterio de verificación inequívoco y comprobable.
- **Agrupación en Hitos (Milestones):**
 1. *Hito 1:* Scaffolding, Tipos Estrictos y Gestión de Estado.
 2. *Hito 2:* Motor de Previsualización y Plantillas A4 Desacopladas.
 3. *Hito 3:* Formularios Modulares del Editor (UI Reactiva).
 4. *Hito 4:* Motor de Impresión y Exportación a PDF de Alta Fidelidad.
 5. *Hito 5:* Orquestación Final, Pruebas y Validación (QA).

GATE 3: CONGELAMIENTO DE TAREAS (TASK FREEZE)

El plan de tareas se aprueba antes de emitir la primera línea de código. Queda prohibido alterar tareas en curso sin un proceso formal de control de cambios.

3.6. Fase 4: Implementación Disciplinada

En la Fase 4, los ingenieros o agentes de IA ejecutan tarea por tarea de forma secuencial. Se aplica la **Regla de Tolerancia Cero al Scope Creep**: si durante la codificación se descubre una necesidad no prevista, no se improvisa en el código; se suspende la tarea, se actualiza la especificación en Fase 1/2, se revalida la compuerta y se reanuda la ejecución.

3.7. Fase 5: Validación, Auditoría de Criterios y Cierre

La entrega final se certifica mediante una **Matriz de Trazabilidad de Requisitos (RTM)**, verificando punto a punto que cada criterio *Given-When-Then* de la Fase 1 se cumple de forma empírica en la aplicación en ejecución.

4. Estudio de Caso Práctico: "CV Wizard Open Source"

4.1. El Reto y los Objetivos del Producto

El proyecto **CV Wizard Open Source** (`practica-sdd-cv-maker`) se diseñó como banco de pruebas integral para validar la metodología SDD en una aplicación interactiva exigente: un generador de currículums profesionales con renderizado reactivo inmediato, cambio dinámico de plantillas, persistencia en el navegador y descarga en PDF sin marcas de agua ni servidores de renderizado externos.

4.2. Análisis de Decisiones Arquitectónicas (Fase 2)

Decisión Técnica	Justificación de Ingeniería	Impacto en Calidad
Astro 5 + React 19 (Islands Architecture)	Estructura base estática ultra-ligera en Astro, montando el editor y el visor como una isla de cliente interactiva (<code>client:only="react"</code>).	Cero sobrecarga de servidor, carga casi instantánea y soporte nativo de Vite.
Persistencia Local-First con Debounce (500ms)	Almacenamiento en <code>localStorage</code> mediante un temporizador elástico que cancela escrituras intermedias durante la pulsación de teclas.	Privacidad 100% garantizada para datos sensibles de CV; cero llamadas de red y resiliencia ante recargas.
Desacoplamiento Estricto de Plantillas	Las plantillas (<code>ModernTemplate</code> , <code>ClassicTemplate</code> , <code>MinimalTemplate</code>) son componentes visuales puros que reciben el objeto inmutable <code>CVData</code> .	Incorporar una 4ª plantilla toma 15 minutos sin alterar el formulario ni la lógica de negocio.
Motor Dual de Impresión Vectorial	En vez de usar librerías pesadas en servidor (como Puppeteer en Node), se formularon reglas CSS avanzadas para el motor de renderizado nativo del navegador.	PDFs vectoriales puros, texto seleccionable, nitidez perfecta y coste computacional cero.

4.3. El Patrón de Desacoplamiento de Plantillas

El mayor logro arquitectónico del proyecto fue aislar la capa de datos de la capa visual. Las tres plantillas implementan la misma firma de componente:

```
interface TemplateProps {
  data: CVData;
}
export const ModernTemplate: React.FC<TemplateProps> = ({ data }) => { ... };
export const ClassicTemplate: React.FC<TemplateProps> = ({ data }) => { ... };
export const MinimalTemplate: React.FC<TemplateProps> = ({ data }) => { ... };
```

4. Estudio de Caso (Cont.): Print Engine y Validación

4.4. Aislamiento Estricto del Motor de Impresión A4 Vectorial

Garantizar que una vista previa en pantalla se traslade milimétricamente a una hoja física A4 (210mm x 297mm) sin elementos de la webapp ni cortes de página requirió reglas precisas en `src/styles/print.css`:

```
@media print {
  @page {
    size: A4 portrait;
    margin: 0mm; /* Elimina márgenes impuestos por el navegador (URLs, fechas) */
  }
  .no-print, .editor-pane, .zoom-controls {
    display: none !important;
  }
  #cv-print-area, .a4-sheet {
    display: flex !important;
    position: relative !important;
    box-shadow: none !important;
    margin: 0 auto !important;
    width: 210mm !important;
    min-height: 297mm !important;
    -webkit-print-color-adjust: exact !important;
    print-color-adjust: exact !important;
  }
}
```

4.5. Matriz de Validación de Criterios de Aceptación (Fase 5)

La aplicación superó el 100% de los criterios definidos en la especificación inicial:

Historia	Criterio de Aceptación Verificado	Mecanismo de Verificación en Código	Estado
US-01	1.1: Formulario modular con actualización instantánea de datos básicos.	<code>ProfileEditor.tsx</code> muta estado reactivo central sin pérdida de foco.	Superado
US-02	2.1: Previsualización con latencia cero en layout dividido (50% / 50%).	<code>ResumeViewer.tsx</code> renderiza hoja A4 sincronizada de forma inmediata.	Superado
US-03	3.1: Intercambio en caliente entre las 3 plantillas preservando datos.	<code>SettingsEditor.tsx</code> altera <code>settings.templateId</code> instantáneamente.	Superado
US-04	4.1: Descarga en PDF A4 vectorial de alta fidelidad sin marcas de agua.	<code>Toolbar.tsx</code> activa <code>window.print()</code> con <code>print.css</code> .	Superado
US-05	5.1: Persistencia en localStorage y exportación/importación JSON.	<code>utils/storage.ts</code> con debounce de 500ms y serializador JSON probado.	Superado

5. SDD en la Era de la Inteligencia Artificial y Agentes Autónomos

5.1. La Paradoja de los Modelos de Lenguaje en Ingeniería

Los agentes de inteligencia artificial (LLMs) son sumamente competentes resolviendo problemas algorítmicos locales, pero sufren de **degradación de contexto y alucinación acumulativa** en desarrollos de mediana o gran escala. Cuando a un modelo se le pide "crear una app", el modelo improvisa librerías, altera contratos de datos y produce inconsistencias que destruyen la arquitectura.

5.2. SDD como Sistema de Control de Vuelo para Agentes IA

Spec-Driven Development actúa como el **marco de confinamiento y anclaje ideal** para agentes como Antigravity:



5.3. Principios Operativos para el Desarrollo con Agentes IA

- **Presupuesto de Ventana de Contexto (Context Budgeting):** Al desacoplar la especificación en archivos compactos (01-specification.md , 02-architecture-design.md , DESIGN.md , 03-tasks.md), el agente solo necesita inyectar en su memoria activa el archivo relevante a la tarea actual, minimizando el consumo de tokens y eliminando alucinaciones.
- **Determinismo mediante Tipos Fuertes:** La obligación de escribir los tipos en TypeScript en Fase 2 actúa como un árbitro estático. El compilador y el linter verifican automáticamente el código generado por la IA.
- **Eliminación del 'Prompt Thrashing':** En lugar de mantener discusiones largas y ambiguas por chat que se diluyen en el contexto, el humano edita el archivo de especificación en Git. La especificación es la única memoria persistente.

6. Marco Operativo: Checklists de Calidad por Fase

Para asegurar la repetibilidad y el control de calidad en futuros proyectos, se establecen las siguientes listas de verificación obligatorias previas a la superación de cada compuerta:

CHECKLIST FASE 1: ESPECIFICACIÓN FUNCIONAL (GATE 1)

- ☐ ¿Se define el problema real del usuario y los diferenciadores clave del sistema?
- ☐ ¿Cada Historia de Usuario cuenta con al menos un criterio formal en formato *Given-When-Then*?
- ☐ ¿Se explicitan los Requisitos No Funcionales (rendimiento, privacidad, latencia, compatibilidad)?
- ☐ ¿El esquema conceptual de datos es agnóstico del framework y está documentado en JSON/TypeScript?
- ☐ ¿Se obtuvo la firma o aprobación formal del cliente/usuario antes de proceder a la arquitectura?

CHECKLIST FASE 2 & 2B: ARQUITECTURA Y SISTEMA DE DISEÑO (GATE 2)

- ☐ ¿Existe una justificación técnica comparativa para cada tecnología seleccionada?
- ☐ ¿El diagrama de flujo de datos modela de forma unívoca la mutación y persistencia del estado?
- ☐ ¿Todas las entidades de datos cuentan con contratos de tipos estrictos sin uso de `any`?
- ☐ ¿El archivo `DESIGN.md` contiene tokens YAML ejecutables para colores, fuentes y radios?
- ☐ ¿Se verificó la accesibilidad de contraste de color bajo el estándar WCAG AA?

CHECKLIST FASE 3: DESGLOSE DE TAREAS ATÓMICAS (GATE 3)

- ☐ ¿Cada tarea está organizada dentro de un Hito cronológico y con dependencias declaradas?
- ☐ ¿Cada tarea incluye una *Definition of Done (DoD)* con criterio de verificación comprobable?
- ☐ ¿Existe trazabilidad explícita entre cada tarea y una Historia de Usuario de la Fase 1?
- ☐ ¿El plan de tareas fue revisado y aprobado antes de emitir cualquier comando de implementación?

6. Marco Operativo (Cont.): Las 10 Reglas de Oro de SDD

1. **Regla de la Página en Blanco:** Si un requisito no está escrito en la especificación, no existe para el sistema.
2. **Regla de No Compensación en Código:** Nunca compenses una ambigüedad de diseño escribiendo código provisional; detente y aclara la especificación.
3. **Regla del Tipado Previo:** Ningún componente visual o servicio de lógica de negocio se codifica antes de que sus tipos de entrada y salida estén completamente definidos.
4. **Regla de la Trazabilidad Inversa:** Toda función, componente o endpoint debe poder justificar su existencia vinculándose a una tarea del plan de ejecución.
5. **Regla del 'Freeze' de Tareas:** Las tareas de un hito no se modifican durante su ejecución. Nuevos descubrimientos generan un proceso formal de actualización de Fase 1.
6. **Regla de la UI Tipográfica y Tokenizada:** No uses valores hexadecimales o medidas mágicas de píxeles arbitrarias; consume exclusivamente los tokens de `DESIGN.md`.
7. **Regla de la Independencia de Plantillas:** La capa de presentación debe ser un consumidor pasivo e intercambiable del modelo de dominio central.
8. **Regla de la Persistencia Elástica:** Toda operación de almacenamiento intensiva en cliente debe aislarse con técnicas de amortiguación (debounce/throttle) para preservar la fluidez de la UI.
9. **Regla de la Prueba de Aceptación:** Un criterio Given-When-Then no se da por cerrado hasta que se demuestra su cumplimiento con datos reales en el entorno destino.
10. **Regla del Guardarraíl Humano (Human-in-the-Loop):** La inteligencia artificial propone planes y ejecuta tareas atómicas, pero la validación de compuertas pertenece indefectiblemente al juicio humano.

Plantilla Estándar de Matriz de Trazabilidad (RTM Template)

ID Requisito	Historia de Usuario (Fase 1)	Componente / Módulo (Fase 2)	ID Tarea (Fase 3)	Prueba de QA (Fase 5)
REQ-01	US-01 (Formulario Modular)	editor/ProfileEditor.tsx	TASK-3.1	Test E2E Form Input
REQ-02	US-02 (Preview Latencia Cero)	preview/ResumeViewer.tsx	TASK-2.1	Benchmark Render Latency
REQ-03	US-03 (Catálogo Plantillas)	preview/templates/*	TASK-2.2 / 2.3 / 2.4	Visual Regression Test
REQ-04	US-04 (PDF Vectorial Sin Marca)	styles/print.css	TASK-4.1	A4 Geometry Print Audit
REQ-05	US-05 (Persistencia Local)	utils/storage.ts	TASK-1.4	Storage Debounce Test

7. Conclusiones y Hoja de Ruta para Nuevos Proyectos

7.1. Síntesis y Valor Estratégico

La construcción de **CV Wizard Open Source** demuestra empíricamente que invertir tiempo en la elaboración de especificaciones estructuradas (Fases 1, 2, 2B y 3) no representa una sobrecarga burocrática, sino un **multiplicador de velocidad y calidad en la fase de implementación**.

Al separar el proceso intelectual de concepción arquitectónica de la tarea mecánica de redacción de sintaxis, los errores de diseño se detectan en minutos sobre documentos de texto, en lugar de tras semanas de depuración de código acoplado.

7.2. Hoja de Ruta para Adoptar SDD en tu Organización

Para incorporar de forma inmediata este marco de referencia en cualquier proyecto nuevo o existente:

1. **Crear el Directorio Canónico:** Establece la carpeta `specs/` en la raíz del repositorio y vincula el archivo `DESIGN.md` para diseño visual.
2. **Adoptar la Plantilla de 3 Documentos:**
 - `specs/01-specification.md`: Visión, Historias de Usuario con GWT y Modelo Conceptual de Dominio.
 - `specs/02-architecture-design.md`: Stack tecnológico, Flujo de Datos, Tipos TypeScript y Directorios.
 - `specs/03-tasks.md`: Desglose cronológico de hitos y tareas atómicas trazables.
3. **Institucionalizar los Approval Gates:** Ningún miembro del equipo (humano o agente IA) debe fusionar código que no pertenezca a una tarea de una especificación debidamente aprobada.
4. **Automatizar la Validación de Tokens:** Integra linters de diseño visual en tus pipelines de Integración Continua (CI) para asegurar fidelidad estética desde el primer commit.

DICTAMEN FINAL DE INGENIERÍA

Spec-Driven Development devuelve a la ingeniería de software su vocación fundacional: **el diseño metódico, el rigor conceptual, la verificabilidad matemática de contratos y la predictibilidad en la entrega**. En conjunción con las capacidades de aceleración de los agentes de IA, SDD constituye el estándar de referencia más sólido y confiable para la construcción de software en el siglo XXI.